

Team 7 - E-Waste Battery Extraction

CV-17: Pixel-to-World Localisation & Validation

Full Technical Implementation Report

ROS 2 Jazzy, Gazebo, Camera Geometry, TF and OpenCV

Ubuntu 24.04 on WSL2

Senior Capstone - Sprint 2 Technical Contribution

September 2026

Contents

1 Task Overview	3
2 System and Experimental Environment	3
3 Part A - Camera Interface and Intrinsic	3
3.1 Overhead Camera Interface	3
4 Part A - Camera Extrinsic and ROS TF	4
4.1 Initial TF Problem	4
4.2 Axis Mapping	5
5 Part A - Pixel-to-World Geometry	5
5.1 Optical Ray Construction	5
5.2 Rotation into the World Frame	6
5.3 Ray-Plane Intersection	6
5.4 Final Geometry Interface	6
6 ROS 2 Integration	7
7 Part A Sanity Check	8
8 Debugging and Design Corrections	8
8.1 Removing the Hard-Coded Camera Pose	8
8.2 Incorrect YOLO Localisation Target	8
8.3 Camera Stream Timeout	9
9 Part B - Validation Method	9
9.1 Why Manual Clicking Was Not Used for the Final Experiment	9
9.2 Automatic Battery Centroid	10
9.3 Validation Positions	10
10 Part B - Localisation Results	11
11 Interpretation	13
12 Limitations and Future Integration	13
13 Final Technical Contribution	14
A Core Commands	14

A.1	Simulation	14
A.2	Localisation TF	14
A.3	Perception	14
A.4	Camera Calibration Check	15
A.5	TF Check	15
A.6	Automatic Validation	15
B	Generated Validation Artefacts	15

1 Task Overview

CV-17 required the perception pipeline to convert image-space object centroids into Gazebo world coordinates using the overhead camera geometry, and then validate the predicted positions against Gazebo ground truth. The work was split into two practical parts. Part A established the localisation geometry and ROS 2 integration. Part B designed a reproducible validation experiment and quantified localisation error across multiple workspace positions.

The final pipeline is:

$$\begin{aligned} \text{camera image} &\rightarrow (u, v) \rightarrow \text{optical-frame ray} \rightarrow \text{TF rotation} \\ &\rightarrow \text{ray-plane intersection} \rightarrow (X, Y, Z)_{\text{world}} \rightarrow \text{ground-truth comparison.} \end{aligned}$$

The main outcome is a TF-based localisation method that no longer depends on a hard-coded camera pose. Camera intrinsics are read from ROS `CameraInfo`, camera extrinsics are provided through ROS TF, and the world-coordinate result is published through the existing detection interface.

2 System and Experimental Environment

The supplied Team 7 project was run in Ubuntu 24.04 under WSL2 with ROS 2 Jazzy and Gazebo. The project workspace was:

```
~/team7_project_bundle/team7_perception_ws
```

The simulation was started with:

```
bash ~/team7_project_bundle/run_team7_sim.sh
```

The preflight verification completed with 20 passed checks and no failures. The relevant perception topics were:

```
/top_camera/image
/top_camera/camera_info
/phone_detections
```

A four-terminal layout was used throughout development:

Table 1: Terminal layout used for CV-17 development and validation.

Terminal	Process	Purpose
1	Team 7 Gazebo simulation	World, camera and test object
2	<code>localisation_tf.launch.py</code>	Permanent camera-to-world transform
3	Perception node	ONNX detector and ROS output
4	Validation commands/scripts	TF, ground truth, experiments and plots

3 Part A - Camera Interface and Intrinsics

3.1 Overhead Camera Interface

The overhead camera publishes a 1280 by 960 image and a matching `CameraInfo` message. The live calibration reported:

$$f_x = f_y = 1108.7654113769531, \quad c_x = 640.0, \quad c_y = 480.0.$$

Therefore the intrinsic matrix is:

$$K = \begin{bmatrix} 1108.7654 & 0 & 640 \\ 0 & 1108.7654 & 480 \\ 0 & 0 & 1 \end{bmatrix}.$$

The distortion model is `plumb_bob` and all distortion coefficients are zero in the Gazebo simulation, so an additional undistortion stage is not required for this camera.

```
ad@DESKTOP-0T45K60:~/team7_project_bundle/team7_perception_ws$ ros2 topic echo /top_camera/camera_info --once
header:
  stamp:
    sec: 352
    nanosec: 0
  frame_id: overhead_camera/mount/overhead_camera
height: 960
width: 1280
distortion_model: plumb_bob
d:
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
k:
- 1108.7654113769531
- 0.0
- 640.0
- 0.0
- 1108.7654113769531
- 480.0
- 0.0
- 0.0
- 1.0
r:
- 1.0
- 0.0
- 0.0
- 0.0
- 1.0
- 0.0
- 0.0
- 0.0
- 1.0
```

Figure 1: Live `/top_camera/camera_info` output confirming resolution and camera intrinsics.

4 Part A - Camera Extrinsics and ROS TF

4.1 Initial TF Problem

The camera bridge stamped the image messages with a camera frame ID, but that frame did not initially exist in the ROS TF tree. A direct `tf2_echo` therefore failed. This prevented a clean geometry implementation from obtaining the camera pose dynamically.

The verified overhead-camera pose in the Gazebo world is:

$$(C_x, C_y, C_z) = (0.0, 0.60, 1.90) \text{ m}$$

with orientation approximately:

$$(\text{roll}, \text{pitch}, \text{yaw}) = (-\pi, 0, -\pi/2).$$

A permanent static TF launch file was then added to provide:

```
world <- top_camera_optical_frame
```

The verified transform reported translation $[0.000, 0.600, 1.900]$ and RPY approximately $[-180, 0, -90]$ degrees.

```
===== BATTERY_1 GROUND TRUTH =====
Requesting state for world [ewaste_room]...

Model: [28]
- Name: battery_1
- Pose [ XYZ (m) ] [ RPY (rad) ]:
  [-0.149998 0.619999 0.800000]
  [-0.000004 -0.000014 0.000173]

===== LOCALISATION TF =====
[INFO] [1788612286.420669975] [tf2_echo]: Waiting for transform world -> top_camera_optical_frame: Invalid frame ID "world" passed to canTransform argument
target_frame - frame does not exist
At time 0.0
- Translation: [0.000, 0.600, 1.900]
- Rotation: in Quaternion (xyzw) [-0.707, 0.707, -0.000, 0.000]
- Rotation: in RPY (radian) [-3.142, -0.000, -1.571]
- Rotation: in RPY (degree) [-180.000, -0.000, -90.000]
- Matrix:
  -0.000 -1.000 0.000 0.000
  -1.000 0.000 -0.000 0.600
  0.000 -0.000 -1.000 1.900
  0.000 0.000 0.000 1.000
At time 0.0
- Translation: [0.000, 0.600, 1.900]
- Rotation: in Quaternion (xyzw) [-0.707, 0.707, -0.000, 0.000]
- Rotation: in RPY (radian) [-3.142, -0.000, -1.571]
- Rotation: in RPY (degree) [-180.000, -0.000, -90.000]
- Matrix:
  -0.000 -1.000 0.000 0.000
  -1.000 0.000 -0.000 0.600
  0.000 -0.000 -1.000 1.900
  0.000 0.000 0.000 1.000
```

Figure 2: Baseline Gazebo ground truth for `battery_1` together with the verified localisation TF.

4.2 Axis Mapping

The transform rotation matrix is approximately:

$$R_{wc} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}.$$

This gives the optical-to-world axis relationship:

$$+x_{optical} \rightarrow -Y_{world}, \quad +y_{optical} \rightarrow -X_{world}, \quad +z_{optical} \rightarrow -Z_{world}.$$

This independently confirmed the axis convention that had been inferred from the original hard-coded geometry.

5 Part A - Pixel-to-World Geometry

5.1 Optical Ray Construction

For an image centroid (u, v) , the standard ROS optical-frame ray is:

$$\mathbf{d}_c = \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}. \quad (1)$$

This uses the ROS optical convention: positive x to the right, positive y down, and positive z forward.

5.2 Rotation into the World Frame

The TF quaternion is converted to a rotation matrix and applied to the optical ray:

$$\mathbf{d}_w = R_{wc} \mathbf{d}_c. \quad (2)$$

The camera origin in world coordinates is:

$$\mathbf{C} = \begin{bmatrix} C_x \\ C_y \\ C_z \end{bmatrix}. \quad (3)$$

The world-space ray is then:

$$\mathbf{P}(t) = \mathbf{C} + t \mathbf{d}_w. \quad (4)$$

5.3 Ray-Plane Intersection

For a horizontal work/object plane at $Z = Z_p$, the intersection parameter is:

$$t = \frac{Z_p - C_z}{d_{w,z}}. \quad (5)$$

The predicted world point is:

$$X = C_x + t d_{w,x}, \quad (6)$$

$$Y = C_y + t d_{w,y}, \quad (7)$$

$$Z = Z_p. \quad (8)$$

The implementation rejects the result when the ray is effectively parallel to the plane or when the intersection lies behind the camera.

5.4 Final Geometry Interface

The active package function is:

```
pixel_to_world_tf(
    u, v,
    fx, fy, cx, cy,
    plane_z,
    camera_pos,
    camera_quat
)
```

and returns:

```
(world_x, world_y, world_z, valid)
```

The active function was inspected directly from the built ROS package before Part B testing.

```

ad@DESKTOP-0T45K60:~/team7_project_bundle/team7_perception_ws$ python3 - <<'PY'
import inspect
import team7_perception.geometry as g

print("==== ACTIVE GEOMETRY FILE =====")
print(g.__file__)

print("\n==== FUNCTION SIGNATURE =====")
print(inspect.signature(g.pixel_to_world_tf))

print("\n==== ACTIVE FUNCTION =====")
print(inspect.getsource(g.pixel_to_world_tf))
PY
==== ACTIVE GEOMETRY FILE =====
/home/ad/team7_project_bundle/team7_perception_ws/build/team7_perception/team7_perception/geometry.py

==== FUNCTION SIGNATURE =====
(u, v, fx, fy, cx, cy, plane_z, camera_pos, camera_quat)

==== ACTIVE FUNCTION =====
def pixel_to_world_tf(
    u,
    v,
    fx,
    fy,
    cx,
    cy,
    plane_z,
    camera_pos,
    camera_quat,
):
    """Project pixel (u,v) onto the world plane z=plane_z using ROS TF.

    camera_pos:
        Camera optical-frame origin expressed in world coordinates (x,y,z).

    camera_quat:
        Quaternion (x,y,z,w) from the TF transform returned by
        lookup_transform(world_frame, camera_optical_frame, ...)

```

Figure 3: Inspection of the active TF-based geometry function used by the running package.

6 ROS 2 Integration

The following package files were modified or added during Part A:

```

team7_perception_ws/src/team7_perception/
|-- launch/
| '-- localisation_tf.launch.py
|-- team7_perception/
| |-- geometry.py
| '-- onnx_phone_detector_node.py
|-- setup.py
'-- package.xml

```

The detector node was updated to:

1. subscribe to /top_camera/camera_info;
2. create a TF buffer and listener;
3. look up world <- top_camera_optical_frame;
4. pass the centroid, intrinsics and TF into pixel_to_world_tf;
5. publish world_x, world_y, world_z and world_valid in the custom detection message.

The workspace was rebuilt with:

```
colcon build --symlink-install
```

and the modified Python files passed syntax checks with `python3 -m py_compile`.

7 Part A Sanity Check

A known raw-image location for the green **battery_1** was used to confirm the geometry numerically. The manually estimated centroid was approximately (619, 632). With the live intrinsics, the verified camera transform, and the settled battery-centre plane $Z = 0.800$ m, the active function produced:

$$(X, Y, Z)_{pred} = (-0.150798, 0.620834, 0.800000) \text{ m}$$

against live Gazebo ground truth:

$$(X, Y, Z)_{GT} = (-0.149998, 0.619999, 0.800000) \text{ m}.$$

The planar error was 1.16 mm. This was treated only as a deterministic geometry sanity check, not as the final localisation benchmark.

```
cy = 488.0

# Battery centroid estimated from the raw image in Part A
u = 619.0
v = 632.0

# Verified world <- top_camera_optical_frame TF
camera_pos = (0.0, 0.600, 1.900)
camera_quat = (-0.70710678, 0.70710678, 0.0, 0.0)

# Current live Gazebo battery centre
gt_x = -0.149998
gt_y = 0.619999
gt_z = 0.800000

wx, wy, wz, valid = pixel_to_world_tf(
    u, v,
    fx, fy, cx, cy,
    gt_z,
    camera_pos,
    camera_quat
)

error_m = hypot(wx - gt_x, wy - gt_y)

print("==== CV-17 PART B BASELINE =====")
print(f"Pixel centroid : ({u:.1f}, {v:.1f})")
print(f"Ground truth   : ({gt_x:.6f}, {gt_y:.6f}, {gt_z:.6f}) m")
print(f"Prediction      : ({wx:.6f}, {wy:.6f}, {wz:.6f}) m")
print(f"Valid           : {valid}")
print(f"XY error         : {error_m:.6f} m")
print(f"XY error         : {error_m * 1000.0:.2f} mm")
PY
==== CV-17 PART B BASELINE =====
Pixel centroid : (619.0, 632.0)
Ground truth   : (-0.149998, 0.619999, 0.800000) m
Prediction      : (-0.150798, 0.620834, 0.800000) m
Valid           : True
XY error         : 0.001157 m
XY error         : 1.16 mm
ad@DESKTOP-0T45K60:~/team7_project_bundle/team7_perception_ws$
```

Figure 4: Baseline Part A/Part B sanity check using the active geometry function.

8 Debugging and Design Corrections

8.1 Removing the Hard-Coded Camera Pose

The original geometry embedded the camera world position and axis mapping directly in code. This worked only while the simulation camera remained at that exact pose. The final implementation instead obtains camera extrinsics from ROS TF, making the geometry explicit and reusable.

8.2 Incorrect YOLO Localisation Target

The live detector produced a high-confidence battery-class detection with a very large bounding box and centroid near (1078.5, 464.5). The resulting world coordinate was far from **battery_1**. This was identified as an upstream detection/localisation association issue rather than a failure of the projection geometry.

```

ad@DESKTOP-OT45K60:~/team7_project_bundle/team7_perception_ws$ timeout 10 ros2 topic echo /phone_detections --once
header:
  stamp:
    sec: 696
    nanosec: 0
  frame_id: overhead_camera/mount/overhead_camera
detections:
- header:
  stamp:
    sec: 696
    nanosec: 0
  frame_id: overhead_camera/mount/overhead_camera
  class_id: battery
  score: 0.9073662757873535
  bbox_x1: 880
  bbox_y1: 19
  bbox_x2: 1278
  bbox_y2: 911
  mar_center_x: 1078.5
  mar_center_y: 464.49993896484375
  mar_width: 396.99993896484375
  mar_height: 890.9998779296875
  angle_deg: 90.0
  world_x: 0.015796911411798856
  world_y: 0.15310204041736597
  world_z: 0.77
  world_valid: true
---
ad@DESKTOP-OT45K60:~/team7_project_bundle/team7_perception_ws$

```

Figure 5: Live detection message showing the oversized battery bounding box that was deliberately excluded from geometry validation.

This distinction became important in Part B: detector error and geometric projection error were evaluated separately rather than allowing a bad detector centroid to contaminate the geometry benchmark.

8.3 Camera Stream Timeout

During repeated Part B tests, `/top_camera/camera_info` continued to publish while `/top_camera/image` temporarily produced no frames within a short 6-10 second window. The validator timeout was increased from 10 seconds to 180 seconds to tolerate the lumpy WSL/Gazebo camera stream without restarting the simulation unnecessarily.

```

ad@DESKTOP-OT45K60:~/team7_project_bundle/team7_perception_ws$ timeout 6 ros2 topic hz /top_camera/image
WARNING: topic [/top_camera/image] does not appear to be published yet
ad@DESKTOP-OT45K60:~/team7_project_bundle/team7_perception_ws$ timeout 6 ros2 topic hz /top_camera/camera_info
average rate: 1.030
  min: 0.895s max: 1.047s std dev: 0.07574s window: 2
ad@DESKTOP-OT45K60:~/team7_project_bundle/team7_perception_ws$ ros2 topic list | grep top_camera
/top_camera/camera_info
/top_camera/image
ad@DESKTOP-OT45K60:~/team7_project_bundle/team7_perception_ws$

```

Figure 6: Debug evidence showing that the camera topic existed while image frames temporarily stalled.

9 Part B - Validation Method

9.1 Why Manual Clicking Was Not Used for the Final Experiment

A manual click validator was first implemented to select the visual battery centre from the live overhead image. The tool successfully demonstrated the complete projection and error calculation pipeline, but repeated clicks at the same object could vary by several pixels and therefore several millimetres.

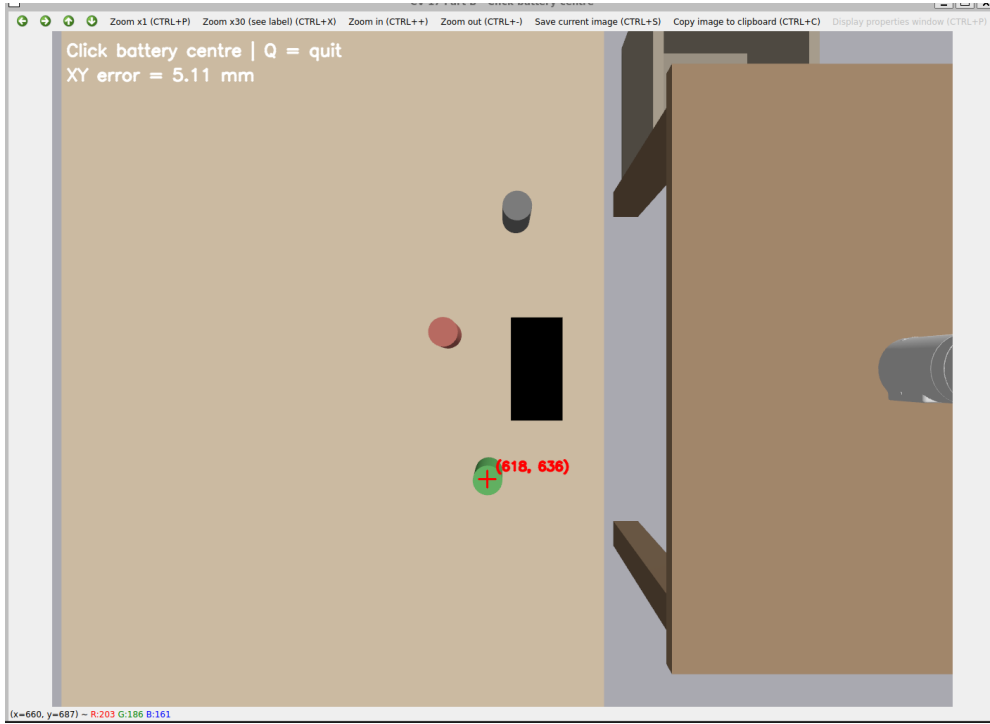


Figure 7: Manual-centroid pilot test. The method worked, but click placement introduced avoidable human uncertainty.

To make the formal experiment reproducible, the final validator used independent green-colour segmentation instead of manual clicking or YOLO output.

9.2 Automatic Battery Centroid

The final validation script converts the BGR image to HSV, thresholds the green battery, applies morphological opening and closing, selects the largest sufficiently large green contour, and calculates a minimum-area rectangle (MAR). The MAR centre is used as the image-space battery centre. The contour-moment centroid is also calculated as a diagnostic check.

This produces a repeatable validation path:

raw camera image $\rightarrow$ green segmentation $\rightarrow$ MAR centre $\rightarrow$ Part A geometry $\rightarrow$ world prediction.

Gazebo ground truth is queried only after the prediction is calculated, so the ground-truth X and Y values are not used as inputs to the projection.

9.3 Validation Positions

Five settled battery positions were used. P2-P5 cover the corners of a rectangular validation region, while P1 provides an interior reference position.

Table 2: Formal CV-17 validation positions.

Position	Ground-truth X (m)	Ground-truth Y (m)	Ground-truth Z (m)
P1	-0.149998	0.619999	0.800000
P2	-0.350000	0.849993	0.800000
P3	0.100002	0.850006	0.800000
P4	0.100003	0.500019	0.800000
P5	-0.349999	0.499996	0.800000

For each position the same algorithm and fixed battery-centre plane $Z_p = 0.800$ m were used.

10 Part B - Localisation Results

The planar error for each sample was calculated as:

$$e_i = \sqrt{(X_{pred,i} - X_{GT,i})^2 + (Y_{pred,i} - Y_{GT,i})^2}. \quad (9)$$

Table 3: Final five-position localisation results.

Pos.	Pred. X (m)	Pred. Y (m)	dX (mm)	dY (mm)	Error (mm)
P1	-0.150798	0.621330	-0.80	+1.33	1.55
P2	-0.350515	0.852030	-0.51	+2.04	2.10
P3	0.100301	0.852190	+0.30	+2.18	2.20
P4	0.101442	0.499550	+1.44	-0.47	1.51
P5	-0.350954	0.499143	-0.96	-0.85	1.28

The aggregate statistics calculated directly from the saved CSV are:

Mean XY error = 1.731 mm,

RMSE = 1.767 mm,

Minimum error = 1.281 mm,

Maximum error = 2.205 mm,

$\overline{\Delta X} = -0.107$ mm,

$\overline{\Delta Y} = +0.846$ mm.

```

===== CV-17 PART B FINAL VALIDATION =====
Formal measurements: 5

Pos      GT X      GT Y      Pred X      Pred Y      dX mm      dY mm      Error mm
P1      -0.149998   0.619999  -0.150798   0.621330    -0.80       1.33       1.55
P2      -0.350000   0.849993  -0.350515   0.852030    -0.51       2.04       2.10
P3       0.100002   0.850006   0.100301   0.852190     0.30       2.18       2.20
P4       0.100003   0.500019   0.101442   0.499550     1.44      -0.47       1.51
P5      -0.349999   0.499996  -0.350954   0.499143    -0.96      -0.85       1.28

===== AGGREGATE STATISTICS =====
Mean XY error : 1.731 mm
RMSE          : 1.767 mm
Minimum error : 1.281 mm
Maximum error : 2.205 mm
Mean dX bias  : -0.107 mm
Mean dY bias  : +0.846 mm

Summary saved to: /home/ad/team7_project_bundle/team7_perception_ws/cv17_validation_summary.csv

```

Figure 8: Final reproducible statistics generated directly from the validation CSV.

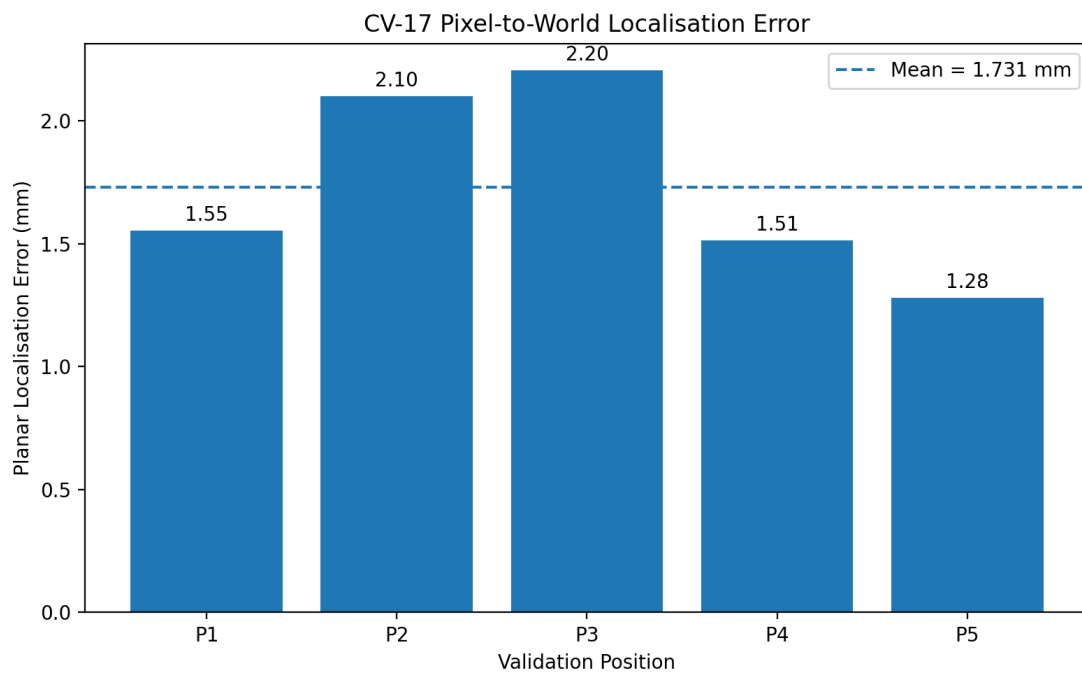


Figure 9: Planar localisation error at the five validation positions.

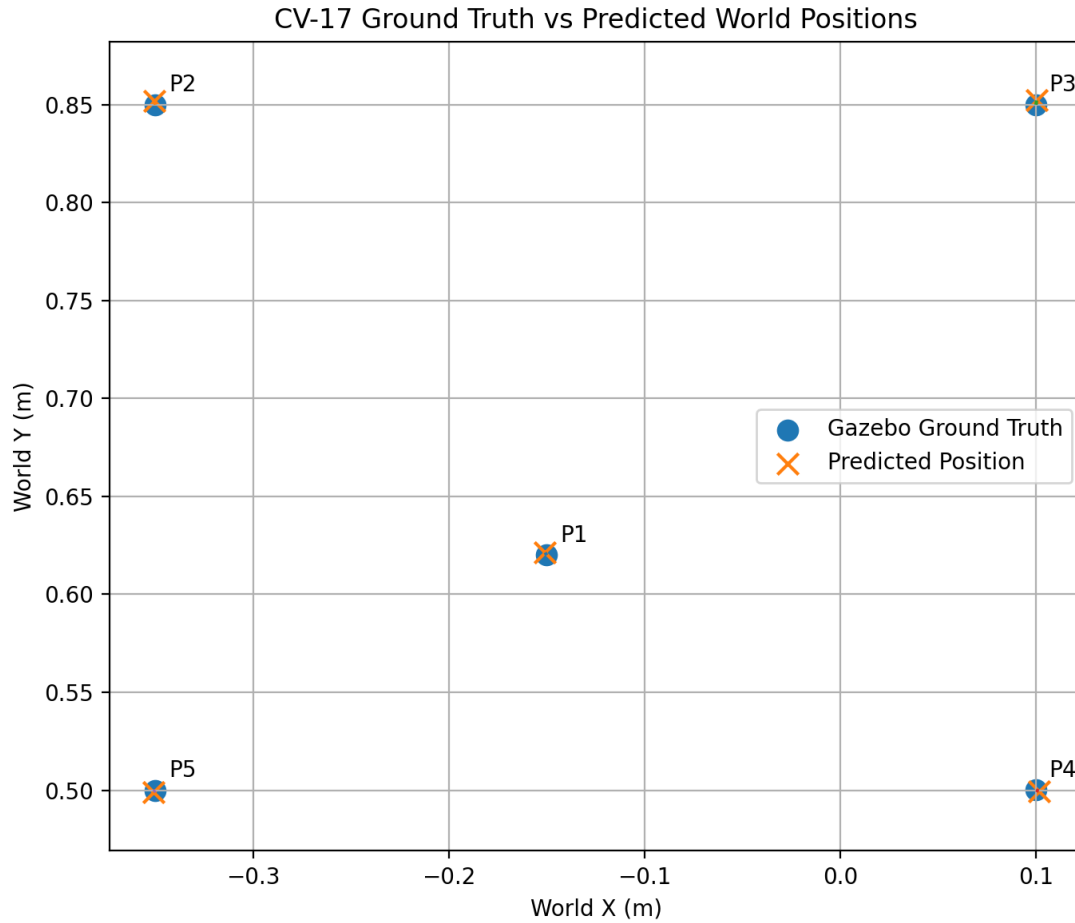


Figure 10: Gazebo ground truth and predicted world positions. The markers nearly overlap at all five test positions.

11 Interpretation

The five-position experiment shows that the implemented camera geometry is consistent across the tested workspace rather than being correct only at one central point. All formal planar errors were below 2.21 mm, with a mean of 1.731 mm. The signed bias is small in both axes, which gives no indication of a large systematic axis or scale error.

The result should be interpreted as the accuracy of the **pixel-to-world geometry under a controlled centroid extraction method**. It is not the end-to-end accuracy of the current YOLO detector because the live detector was known to produce an incorrect bounding region for the battery during this work.

Because the validation plane was fixed at the settled battery-centre height $Z = 0.800$ m, the experiment validates planar localisation on that horizontal plane. Depth was not independently estimated. For this reason, XY error is the primary metric rather than a claimed general 3-D localisation error.

12 Limitations and Future Integration

The main remaining limitations are:

- the current YOLO battery-class localisation needs separate correction before end-to-end detector-

to-world accuracy can be reported;

- the present validation assumes a known horizontal target plane;
- the camera is fixed in the Gazebo simulation; a physical camera would require a calibrated and maintained TF relationship;
- the colour-segmentation validator is an experimental ground-truth aid, not a production perception component;
- the test uses one battery model class and five controlled positions rather than a large stochastic benchmark.

These limitations do not invalidate the CV-17 geometry result. They define the boundary between verified localisation geometry and later end-to-end perception work such as CV-18 and downstream planning integration.

13 Final Technical Contribution

CV-17 Part A and Part B together produced a complete robot-perception localisation contribution rather than a detector-only feature. The work established the mathematical projection, integrated it with ROS 2 TF and camera calibration, exposed world coordinates through the existing message interface, designed a reproducible Gazebo validation method, and quantified accuracy across the workspace.

The final contribution can be summarised as:

A ROS 2 pixel-to-world localisation pipeline was implemented using live camera intrinsics, a verified world-to-optical-frame TF, quaternion-based ray transformation and ray-plane intersection. The implementation was then validated against Gazebo ground-truth object poses at five workspace positions, producing a mean planar error of 1.731 mm and a maximum error of 2.205 mm.

A Core Commands

A.1 Simulation

```
bash ~/team7_project_bundle/run_team7_sim.sh
```

A.2 Localisation TF

```
cd ~/team7_project_bundle/team7_perception_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch team7_perception localisation_tf.launch.py
```

A.3 Perception

```
bash ~/team7_project_bundle/run_team7_perception.sh
```

A.4 Camera Calibration Check

```
ros2 topic echo /top_camera/camera_info --once
```

A.5 TF Check

```
ros2 run tf2_ros tf2_echo world top_camera_optical_frame
```

A.6 Automatic Validation

```
python3 cv17_validate_green.py
```

B Generated Validation Artefacts

The final Part B run generated:

```
cv17_validation_auto_results.csv  
cv17_validation_summary.csv  
cv17_validation_evidence/  
  cv17_auto_*.png  
  cv17_localisation_error_by_position.png  
  cv17_ground_truth_vs_prediction.png
```

These files provide the reproducible numerical and visual evidence for the final CV-17 validation result.